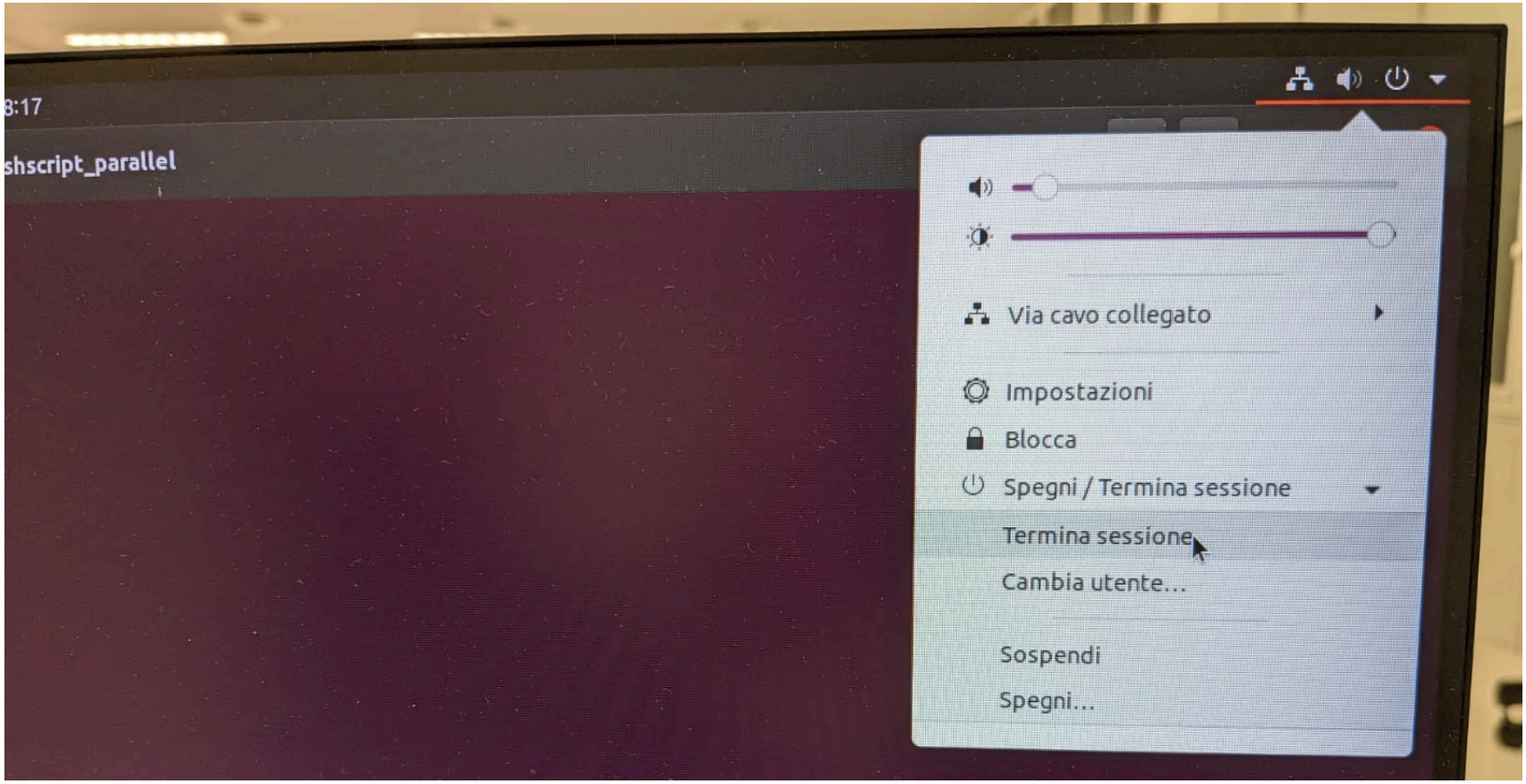
Corso di Sistemi Operativi

Corso estinto di Sistemi Operativi e Reti - Modulo Sistemi Operativi

Prova scritta - Gennaio 2026

LEGGI ATTENTAMENTE LE ISTRUZIONI:

1. **Rinomina** subito la cartella chiamata "Cognome-Nome-Matricola" che hai trovato sul Desktop e in cui hai trovato questa traccia, sostituendo "Cognome" "Nome" e "Matricola" con i tuoi dati personali e **lasciando i trattini senza spazi**; se hai un doppio nome oppure un doppio cognome dovrai chiamare la cartella come in questo esempio:
    a. DeLuca-MarcoGiovanni-199999
2. **Carica** tutto il materiale didattico che vorrai usare sul Desktop; puoi farlo solo nei primi 5 minuti della prova;
3. **Svolgi** il compito; lascia tutto il sorgente che hai prodotto nella cartella di cui al punto 1;
4. Quando hai finito fai "**Termina Sessione/Logout**", ma lascia la postazione mantenendo il PC acceso.
5. **E' tua diretta responsabilità** garantire l'integrità del tuo elaborato, anche in caso di assenza di corrente. **Salva spesso il tuo lavoro**



e NON spegnere il PC.

SALVA SPESSO

CI SONO DEI PUNTI AMBIGUI NELLA TRACCIA? DECIDI TU COSA FARE

È parte integrante di questo esercizio completare le specifiche date nei punti non esplicitamente definiti, introducendo nuove strutture dati, o estendendo quelle preesistenti laddove si ritenga necessario, risolvendo eventuali ambiguità. Si può cambiare il codice dei metodi esistenti dove serve.

POSSO CAMBIARE IL PROTOTIPO DEI METODI RICHIESTI O DI QUELLI ESISTENTI? Si, ma solo se li lasci retrocompatibili

Il codice che scriverai è pensato per essere usato come un modulo di libreria. Un modulo di libreria potrebbe essere usato da altri programmatori, i quali si aspettano di trovare una specifica interfaccia.

Se quindi modifichi il prototipo dei metodi pubblici di una classe, devi assicurarti che possano essere comunque utilizzati per come erano stati pensati prima delle tue modifiche. Puoi aggiungere qualsivoglia campo e metodo privato, e qualsivoglia classe ausiliaria. Analogamente, i metodi esistenti possono essere modificati nel loro codice, ma non se ne deve cambiare il risultato finale o il significato a meno che non ti venga richiesto esplicitamente.

CHE LINGUAGGIO POSSO USARE? PYTHON 3.X

POSSO CONSENTIRE SITUAZIONI DI RACE CONDITION NEL MIO CODICE? NO
POSSO CONSENTIRE SITUAZIONI DI DEADLOCK NEL MIO CODICE? NO
POSSO CONSENTIRE ALTRE SITUAZIONI DI BLOCCO TOTALE NEL MIO CODICE, TIPO NESTED LOCKOUT, LIVELOCK O ALTRO? NO
POSSO CONSENTIRE SITUAZIONI DI STARVATION NEL MIO CODICE? SI, tranne quando ti viene chiesto esplicitamente di rimuoverle
MA IL MAIN() LO DEVO AGGIORNARE? E I THREAD DI PROVA? SI

E' obbligatorio implementare esplicitamente il codice di prova oppure modificare il codice di prova pre-esistente, e accertarsi che giri senza errori prima della consegna.

ESERCIZIO 1 - PROGRAMMAZIONE MULTITHREADED

(Punteggio minimo richiesto 18/30. Pesa per ⅔ del voto finale)

Svolgi tutti i punti della traccia in un unico file che chiamerai `SOLUZIONE.py`
`-> Qualsiasi altro nome di file verrà IGNORATO <-`

Punto 1

Fai in modo che la scuola a cui richiedere l'iscrizione non venga scelta casualmente. Un genitore dovrà invece scegliere ogni volta la scuola che ha ricevuto il minor numero di richieste di iscrizione. Modifica il codice affinché un genitore continui a tentare di iscrivere i propri figli finché l'iscrizione non vada a buon fine oppure finché non ci siano più scuole disponibili. A tale scopo, nel `SistemaIscrizioni`, aggiungi una lista delle scuole attualmente con il minor numero di iscrizioni. I genitori dovranno scegliere la scuola da tale lista.
(Non devi preoccuparti del fatto che il numero di iscrizioni di una scuola possa cambiare *dopo* che il genitore abbia consultato l'elenco per poter effettuare la propria scelta.)

Punto 2

Ricongiungi eventuali famiglie spezzate. Aggiungi un thread `Segretario` (uno per ogni scuola) che si occupi di cercare eventuali fratelli degli iscritti alla propria scuola che siano iscritti ad una scuola diversa. In particolare il ciclo di vita del thread dovrà essere il seguente: 1) attesa bloccante fino a che la scuola non raggiunga il numero massimo di iscrizioni; 2) per ogni iscritto `I`, cerca eventuali fratelli `F` in altre scuole e scambi un proprio iscritto `I1` (con `I` non fratello di `I1`) con `F`.

Punto 3

Se un figlio dovesse essere coinvolto in uno scambio, il relativo genitore deve esprimere la presa visione di quanto

accaduto. Ad esempio se il figlio `figlio_1` viene trasferito dalla `scuola_1` alla `scuola_2`, il thread genitore dovrà stampare "Trasferimento di `figlio_1` da `scuola_1` a `scuola_2` concluso". Nota che il genitore deve prima iscrivere tutti i figli e solo dopo esprimere le eventuali prese visione. Assicurati che nessun genitore rimanga in attesa di comunicazioni a tempo indeterminato.

SALVA SPESSO

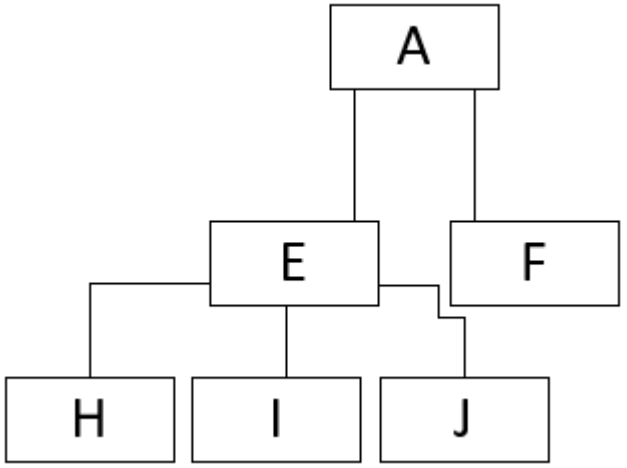
ESERCIZIO 2 - LINGUAGGI DI SCRIPTING

(Punteggio minimo richiesto 18/30. Pesa per ⅓ del voto finale)

NOTA BENE: In questo esercizio dovrai sistemare errori nello script che trovi nella cartella della traccia e non implementarne uno da zero. Leggi la traccia fino in fondo prima di procedere.
CREA TU EVENTUALI FILE DI TESTING.

Lo script `cerca_e_copia.pl` dovrebbe comportarsi nel seguente modo:

1. se eseguito senza opzioni, lo script dovrebbe:
   1.1. cercare tutti i file con estensione ".pl" che si trovano direttamente all'interno di qualche cartella allo stesso livello di quella da cui è stato lanciato lo script `cerca_e_copia.pl`. N.B. ricorda che la cartella dalla quale viene lanciato lo script non è sempre quella in cui si trova lo script stesso. **Ad esempio**, dato l'albero di cartelle qui a destra, supponiamo che lo script `cerca_e_copia.pl` si trovi in `E`. Se lo script viene invocato dalla cartella `E`, poiché l'unica cartella di pari livello di `E` è `F`, i file `.pl` devono essere cercati SOLO in `E` e `F`; se lo script viene invocato da `J`, poiché le cartelle di pari livello di `J` sono `I` e `H`, bisognerà cercare SOLO in `H`, `I` e `J`.



1.2. copiare i file trovati in una cartella chiamata `backup` che verrà creata dallo script `cerca_e_copia.pl` nella cartella dalla quale è stato eseguito lo script stesso (nei due casi dell'esempio precedente, le cartelle di destinazione saranno `E` e `J` rispettivamente). Nel caso in cui lo script si dovesse trovare a copiare due file con lo stesso nome, verrà copiato quello con la data di modifica più recente.

1.3. creare un archivio `backup.tar.gz` contenente la cartella `backup`.

1.4. stampare su STDOUT il numero di file copiati.

2. se eseguito come `perl cerca_e_copia.pl PATH_ASSOLUTO` lo script dovrebbe:

2.1. cercare tutti i file ".pl" che si trovano nella cartella indicata dal path `PATH_ASSOLUTO`.

2.2. eseguire quanto richiesto ai punti 1.2 e 1.3.

Lo script allegato, però, presenta errori di sintassi e/o logici. Sistemalo per ottenere il comportamento sopra descritto. ***NON devi stravolgere il contenuto dello script: la sua struttura deve rimanere pressoché invariata. La correzione di codice che non presenta bug comporta una penalità.***